B+ Trees and Index Files

Lecture : Structure, Searching, Insertion, Deletion and Implementation

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Recap: B-Trees

Today's Question

Can we organize index keys and data references separately to make sequential and range-based access more efficient?

Answer

Yes. B+ trees provide this organization.

What Is a B+ Tree?

Definition

A B+ tree is a balanced multiway search tree in which internal nodes contain separator keys and child pointers, while data-record references are stored at the leaf level.

- All leaves are at the same depth.
- Internal nodes guide searches.
- Leaf nodes contain indexed keys.
- Leaf nodes are commonly linked.
- Supports exact and range searches.

Why B+ Trees?

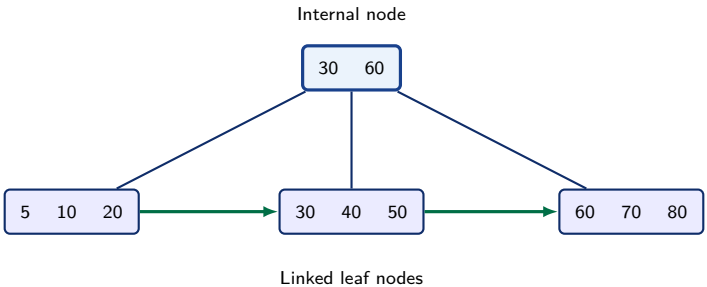
B+ trees are designed for efficient indexing of large collections of records.

1. Internal nodes act as a search directory.
2. Leaf nodes contain the indexed entries.
3. Linked leaves support ordered traversal.
4. High fan-out reduces tree height.
5. Insertions and deletions maintain balance.

Application

Searching student records by roll number, or retrieving all students whose roll numbers lie within a specified range.

B+ Tree: Basic Structure



Observation

Separator keys 30 and 60 appear in the root and in the leaf level.

Internal Nodes

Internal nodes contain:

- Separator keys.
- Pointers to child nodes.
- Information used to select a search path.

For a node with k separator keys, there are normally $k + 1$ child pointers.

Example

$$[30 \mid 60]$$

Child ranges:

$$K < 30$$

$$30 \leq K < 60$$

$$K \geq 60$$

Leaf Nodes

Leaf nodes contain:

- Indexed search keys.
- References to corresponding records.
- A pointer to the next leaf node.



Important

All indexed keys are represented at the leaf level.

Properties of a B+ Tree

1. It is a balanced multiway search tree.
2. All leaves appear at the same depth.
3. Internal nodes contain separator keys.
4. Data references are maintained at leaves.
5. Leaves are arranged in sorted key order.
6. Leaves are commonly linked for sequential access.
7. Insertions may cause node splitting.
8. Deletions may cause redistribution or merging.

Order and Capacity Convention

B+ tree terminology varies between textbooks.
For the examples in this lecture, we use:

Maximum Internal Children

$$4$$

Maximum Keys per Node

$$3$$

- Non-root internal nodes: at least 2 children.
- Non-root leaves: at least 2 keys.
- Root may have fewer keys.

Note

Leaf occupancy conventions may differ between implementations.

Why Are Leaf Nodes Linked?

Consider the query:

$$30 \leq K \leq 70$$

Without leaf links, the search structure would need repeated tree navigation.
With linked leaves:

1. Find the first key greater than or equal to 30.
2. Read the remaining entries in that leaf.
3. Follow the next-leaf pointer.
4. Stop after passing 70.

Benefit

Efficient ordered and range-based retrieval.

Range Search Example



Query

Find all keys in the range:

$$30 \leq K \leq 60$$

Output

$$\boxed{30,\ 40,\ 50,\ 60}$$

B-Tree vs B+ Tree: Main Difference

B-Tree

Keys and associated data references may occur in internal nodes as well as leaves.

B+ Tree

Internal nodes act as a directory.
Data references are stored at the leaf level.

Important

A key may appear both as an internal separator and as a leaf entry in a B+ tree.

B-Tree vs B+ Tree: Comparison

Feature	B-Tree	B+ Tree
Data references	Internal or leaf nodes	Leaf level
Internal nodes	May contain data references	Contain separator keys
Key repetition	Typically one copy	Separator keys may repeat
Search termination	May stop internally	Reaches leaf for data
Sequential access	Less directly supported	Linked leaves help
Range queries	Supported	Especially efficient

Why B+ Trees Are Popular for Indexing

1. Internal nodes can have high fan-out.
2. Searches follow a balanced path.
3. Linked leaves simplify range scans.
4. Records can be accessed in sorted order.
5. Updates preserve the balanced structure.

Example

A database query retrieving roll numbers between 1000 and 2000 can locate the starting leaf and scan forward.

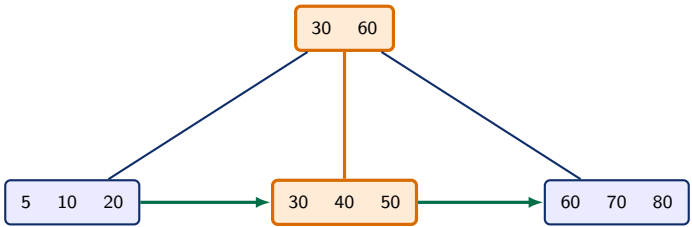
B+ Tree Search Algorithm

1. Start at the root.
2. Compare the target key with separator keys.
3. Choose the appropriate child pointer.
4. Repeat until a leaf node is reached.
5. Search for the key within the leaf.
6. Return the associated record reference.

Important

Internal separator keys guide navigation; the indexed entry is located at the leaf.

Searching: Find Key 50



1. Compare 50 with 30 and 60.
2. Follow the middle child.
3. Search leaf $[30, 40, 50]$.
4. Key 50 is found.

Searching: Find Key 45

Use the same B+ tree.

1. Start at root $[30, 60]$.
2. Since $30 \leq 45 < 60$, select middle child.
3. Visit leaf $[30, 40, 50]$.
4. Compare with the leaf entries.
5. Key 45 does not exist.

Result

Search Unsuccessful

Search Pseudocode

Algorithm: BPlusSearch

```
1.  Set current = root
2.  While current is not a leaf:
    Find the appropriate child index
    Move to that child
3.  Search the leaf keys
4.  If the key exists:
    Return the record reference
5.  Otherwise:
    Return NOT FOUND
```

Insertion Algorithm

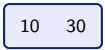
1. Search for the correct leaf.
2. Insert the new key in sorted order.
3. If the leaf has sufficient space, stop.
4. Otherwise, split the leaf.
5. Copy the first key of the new right leaf into the parent as a separator.
6. Split internal nodes if they overflow.
7. Create a new root if necessary.

Important Difference

During a leaf split, the separator is copied upward and remains in the leaf.

Insertion Without Splitting

Maximum keys per leaf $= 3$.
Initial leaf:



Insert 20:



Result

The leaf has three keys.
No splitting is required.

Insertion: Leaf Overflow

Initial leaf:

$$[10,\ 20,\ 30]$$

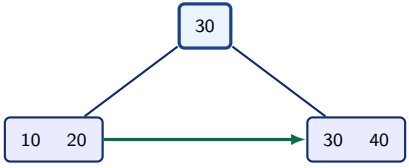
Insert 40:

$$[10,\ 20,\ 30,\ 40]$$

The leaf exceeds its capacity.
Split into:

$$[10,\ 20] \qquad [30,\ 40]$$

Copy separator 30 into the parent.



Worked Example: Insert Keys

Question

Construct a B+ tree with a maximum of three keys per node by inserting:

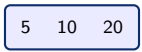
$$\boxed{10,\ 20,\ 5,\ 6,\ 12,\ 30,\ 7,\ 17}$$

Rules:

- Each leaf holds at most three keys.
- Split an overflowing leaf into two halves.
- Copy the first right-leaf key upward.
- Internal nodes hold at most three separators.

Step 1: Insert 10, 20, 5

Insert the first three keys:



Observation

The tree consists of one leaf node, which is also the root.

Step 2: Insert 6

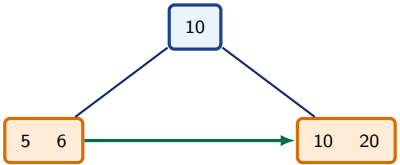
Before insertion:

$$[5,\ 10,\ 20]$$

After insertion:

$$[5,\ 6,\ 10,\ 20]$$

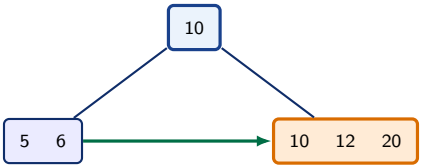
Split the leaf.



Separator 10 is copied into the root.

Step 3: Insert 12

Key 12 belongs in the right leaf.



Observation

The right leaf contains three keys.
No split is required.

Step 4: Insert 30

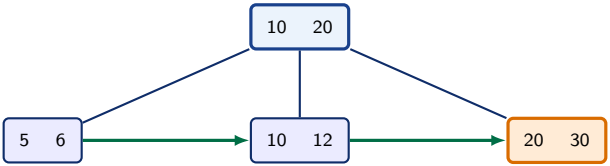
The right leaf overflows:

$$[10,\ 12,\ 20,\ 30]$$

Split:

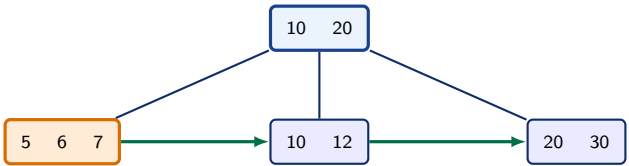
$$[10,\ 12] \quad [20,\ 30]$$

Copy separator 20 into the root.



Step 5: Insert 7

Insert 7 into the leftmost leaf.

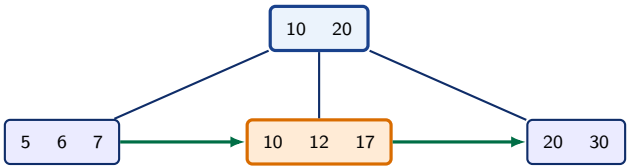


Observation

No split occurs.

Step 6: Insert 17

Insert 17 into the middle leaf.



Final Tree

All eight keys are present at the leaf level.
The root contains separator keys 10 and 20.

When Does an Internal Node Split?

An internal node may hold at most three separator keys in our example.
Suppose it temporarily contains:

$$[20,\ 40,\ 60,\ 80]$$

This exceeds the capacity.

Solution

Split the internal node and promote a separator into its parent.

Unlike a leaf split, the promoted internal separator is removed from the split children.

Internal Node Split: Example

Before splitting:



Promote 60:

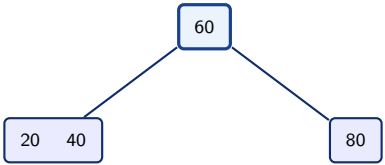


Diagram Note

The lower boxes here illustrate separator distribution only, not actual leaf contents.
The original five child pointers must also be redistributed: three to the left internal node and two to the right.

Leaf Split vs Internal Split

Feature	Leaf Split	Internal Split
Contents	Indexed keys and data references	Separators and child pointers
Promoted key	Copied to parent	Moved to parent
Key remains below?	Yes	Not in split children
Leaf links	Updated	Not applicable
May propagate?	Yes	Yes

Root Splitting

If the root overflows:

1. Split the root into two nodes.
2. Promote an appropriate separator.
3. Create a new root.
4. Attach the two resulting nodes.

Result

The height increases by one.
All leaves remain at the same depth.

B+ Tree Deletion

Deletion removes the indexed entry from the appropriate leaf.

1. Locate the target leaf.
2. Remove the key and its record reference.
3. Check minimum occupancy.
4. Borrow from a sibling if possible.
5. Otherwise merge with a sibling.
6. Update parent separator keys.
7. Continue rebalancing upward if necessary.

Case 1: Simple Deletion

Suppose a leaf contains:

$$[10,\ 20,\ 30]$$

Delete key 20:

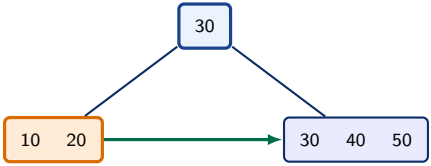
$$[10,\ 30]$$

Result

The leaf still contains two keys, meeting the minimum occupancy used in this lecture.
No structural change is required.

Case 2: Leaf Underflow

Consider:



Delete key 20.
The left leaf becomes:

[10]

Underflow

The leaf contains fewer than two keys.

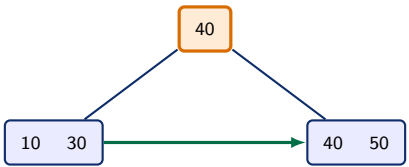
Case 3: Borrowing from a Sibling

The right sibling contains:

$$[30,\ 40,\ 50]$$

Borrow its smallest key, 30.
The resulting leaves are:

$$[10,\ 30] \qquad [40,\ 50]$$

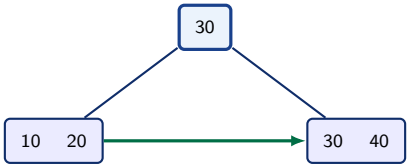


Important

The parent separator changes from 30 to 40.

Case 4: Merging Leaf Nodes

Consider:



Delete 20.
The left leaf underflows.
The right leaf cannot lend a key.
Merge both leaves:

$$[10,\ 30,\ 40]$$

Leaf Merge: Final Result

After merging:



Result

The old root is removed.
The merged leaf becomes the root.
The height decreases by one.

Important

In B+ tree deletion, parent separators must remain consistent with the keys stored in their child subtrees.

Deletion: Key Observations

- Data entries are deleted from leaves.
- Internal separators may need updating.
- Borrowing redistributes keys between leaves.
- Merging combines underfull nodes.
- Leaf links must be maintained.
- Internal underflow may propagate upward.
- Root contraction can reduce tree height.

B+ Tree Implementation in C

The following program implements:

1. Node creation.
2. Searching.
3. Recursive insertion.
4. Leaf splitting.
5. Internal node splitting.
6. Root splitting.
7. Leaf-level traversal.
8. Memory cleanup.

Convention

Maximum keys per node = 3.
Duplicate keys are not inserted.

Scope

Deletion is covered through algorithms and diagrams, not in this C program.

C Program: Headers and Node Structure

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_KEYS 3

typedef struct Node {
    int leaf;
    int n;
    int keys[MAX_KEYS + 1];
    struct Node *child[MAX_KEYS + 2];
    struct Node *next;
} Node;

Node *createNode(int leaf) {
    Node *p = calloc(1, sizeof(Node));

    if (p == NULL) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }

    p->leaf = leaf;
    return p;
}
```

C Program: Search

```c
int search(Node *root, int key) {
    Node *p = root;

    while (p != NULL && !p->leaf) {
        int i = 0;

        while (i < p->n &&
               key >= p->keys[i]) {
            i++;
        }

        p = p->child[i];
    }

    if (p == NULL)
        return 0;

    for (int i = 0; i < p->n; i++) {
        if (p->keys[i] == key)
            return 1;
    }

    return 0;
}
```

Search Rule

A key equal to an internal separator follows the child to its right.

C Program: Splitting a Leaf

```
void splitLeaf(Node *p,
               int *up,
               Node **right) {
    int mid = (p->n + 1) / 2;

    *right = createNode(1);
    (*right)->n = p->n - mid;

    for (int i = 0; i < (*right)->n; i++) {
        (*right)->keys[i] =
            p->keys[mid + i];
    }

    p->n = mid;

    (*right)->next = p->next;
    p->next = *right;

    *up = (*right)->keys[0];
}
```

Key Idea

The first key of the new right leaf is copied into the parent.

C Program: Splitting an Internal Node

```c
void splitInternal(Node *p,
                   int *up,
                   Node **right) {
    int mid = p->n / 2;

    *up = p->keys[mid];
    *right = createNode(0);

    (*right)->n = p->n - mid - 1;

    for (int i = 0; i < (*right)->n; i++) {
        (*right)->keys[i] =
            p->keys[mid + 1 + i];
    }

    for (int i = 0; i <= (*right)->n; i++) {
        (*right)->child[i] =
            p->child[mid + 1 + i];
    }

    p->n = mid;
}
```

C Program: Recursive Insertion, Part 1

```
int insertRec(Node *p, int key,
              int *up, Node **right) {
    int i = 0;

    if (p->leaf) {
        while (i < p->n && p->keys[i] < key)
            i++;

        if (i < p->n && p->keys[i] == key)
            return 0;

        for (int j = p->n; j > i; j--)
            p->keys[j] = p->keys[j - 1];

        p->keys[i] = key;
        p->n++;

        if (p->n <= MAX_KEYS)
            return 2;

        splitLeaf(p, up, right);
        return 1;
    }
```

C Program: Recursive Insertion, Part 2

```
    while (i < p->n && key >= p->keys[i])
        i++;

    int promoted;
    Node *newRight = NULL;

    int status = insertRec(
        p->child[i], key,
        &promoted, &newRight
    );

    if (status != 1)
        return status;

    for (int j = p->n; j > i; j--) {
        p->keys[j] = p->keys[j - 1];
        p->child[j + 1] = p->child[j];
    }

    p->keys[i] = promoted;
    p->child[i + 1] = newRight;
    p->n++;

    if (p->n <= MAX_KEYS)
        return 2;

    splitInternal(p, up, right);
    return 1;
}
```

Status Codes

0 = duplicate, 1 = split, 2 = inserted.

C Program: Root Management

```c
int insert(Node **root, int key) {
    if (*root == NULL) {
        *root = createNode(1);
        (*root)->keys[0] = key;
        (*root)->n = 1;
        return 1;
    }

    int up;
    Node *right = NULL;

    int status = insertRec(
        *root, key, &up, &right
    );

    if (status == 0)
        return 0;

    if (status == 1) {
        Node *newRoot = createNode(0);

        newRoot->keys[0] = up;
        newRoot->n = 1;
        newRoot->child[0] = *root;
        newRoot->child[1] = right;

        *root = newRoot;
    }

    return 1;
}
```

C Program: Leaf Traversal

```c
void printLeaves(Node *root) {
    if (root == NULL)
        return;

    Node *p = root;

    while (!p->leaf)
        p = p->child[0];

    printf("Leaf traversal:\n");

    while (p != NULL) {
        printf("[ ");

        for (int i = 0; i < p->n; i++)
            printf("%d ", p->keys[i]);

        printf("] ");

        p = p->next;
    }

    printf("\n");
}
```

C Program: Memory Cleanup

```
void freeTree(Node *p) {
    if (p == NULL)
        return;

    if (!p->leaf) {
        for (int i = 0; i <= p->n; i++)
            freeTree(p->child[i]);
    }

    free(p);
}
```

Observation

Each node is released once.
The next-leaf pointers are not followed during cleanup because leaf nodes are already reachable through the tree.

C Program: Main Function

```c
int main(void) {
    Node *root = NULL;

    int values[] = {
        10, 20, 5, 6,
        12, 30, 7, 17
    };

    int n = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < n; i++)
        insert(&root, values[i]);

    printLeaves(root);

    printf("Search 17: %s\n",
           search(root, 17) ?
           "Found" : "Not Found");

    printf("Search 99: %s\n",
           search(root, 99) ?
           "Found" : "Not Found");

    freeTree(root);

    return 0;
}
```

Expected Program Output

Output

```
Leaf traversal:
[ 5 6 7 ] [ 10 12 17 ] [ 20 30 ]
Search 17:  Found
Search 99:  Not Found
```

Observation

The leaf traversal produces sorted keys.
Search proceeds through internal nodes and terminates at a leaf.

Time Complexity

For a balanced B+ tree containing n keys:

Operation	Complexity
Search	$O(\log n)$
Insertion	$O(\log n)$
Deletion	$O(\log n)$
Full leaf traversal	$O(n)$

Range Queries

For a range returning k keys:

$$\boxed{O(\log n + k)}$$

assuming fixed node capacity and a linked leaf structure.

Why Is B+ Tree Height Small?

A B+ tree node can have multiple children.
If an internal node supports a large number of child pointers, the tree can index many keys using relatively few levels.

Benefit

Lower height generally reduces the number of index pages read.

Practical Use

B+ trees are widely used in database storage engines and file-system indexing.

Applications of B+ Trees

1. Database indexes.
2. File systems.
3. Ordered key-value stores.
4. Large-scale record retrieval.
5. Range-based database queries.
6. Secondary-storage indexing.

University Example

Find all student records having roll numbers between 2027000 and 2027999.

Advantages of B+ Trees

- Balanced search paths.
- Efficient exact-match queries.
- Efficient sequential access.
- Fast range retrieval.
- High fan-out.
- Good use of disk pages.
- Dynamic insertion and deletion.

Limitations of B+ Trees

- More complex implementation.
- Splitting and merging require maintenance.
- Internal separators duplicate some leaf keys.
- Leaf links must be updated during changes.
- Disk-based versions require page management.

Trade-off

Additional structural maintenance provides efficient balanced access.

Exercise 1: Construction

Question

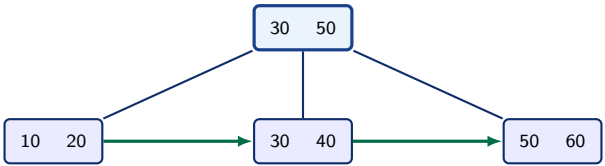
Construct a B+ tree with at most three keys per node by inserting:

$$\boxed{10,\ 20,\ 30,\ 40,\ 50,\ 60}$$

- Show each leaf split.
- Identify copied separators.
- Draw linked leaf nodes.

Exercise 1: Solution

After inserting all six keys:



Verification

All six keys appear in linked leaves.
The root stores separators 30 and 50.

Exercise 2: Range Search

Question

A B+ tree has the following leaf sequence:

$$[5, 10] \rightarrow [15, 20] \rightarrow [25, 30] \rightarrow [35, 40]$$

Retrieve all keys satisfying:

$$18 \leq K \leq 36$$

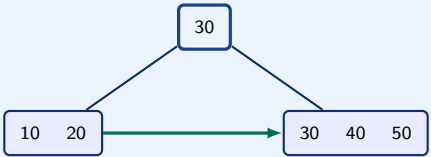
Solution

$$\boxed{20,\ 25,\ 30,\ 35}$$

Exercise 3: Deletion

Question

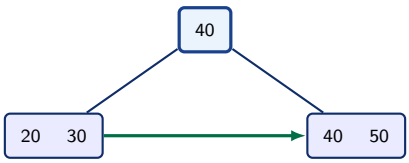
A B+ tree contains:



Delete 10 and rebalance.

Exercise 3: Solution

After deleting 10, borrow 30 from the right leaf.



Result

The separator is updated to 40.
Both leaves satisfy minimum occupancy.

MCQ 1

Where are data-record references stored in a B+ tree?

1. Only in the root
2. At the leaf level
3. Only in internal nodes
4. Outside the tree

Correct Answer

Option 2.

MCQ 2

Which feature supports efficient sequential access in a B+ tree?

1. Linked leaf nodes
2. Binary-only branching
3. Unsorted keys
4. Absence of separators

Correct Answer

Option 1.

MCQ 3

What happens to the separator during a leaf split?

1. It is deleted permanently.
2. It is copied into the parent.
3. All leaf keys move to the root.
4. The tree becomes unbalanced.

Correct Answer

Option 2.

MCQ 4

Which query is particularly well suited to a B+ tree?

1. Range query
2. Random arithmetic computation
3. Matrix multiplication
4. Graph coloring

Correct Answer

Option 1.

MCQ 5

What is the typical search complexity of a balanced B+ tree?

1. $O(n^2)$
2. $O(n)$
3. $O(\log n)$
4. $O(2^n)$

Correct Answer

Option 3.

B+ Tree: Important Concepts

Concept	Description
Internal node	Search separators
Leaf node	Indexed entries
Leaf link	Sequential access
Search	Root to leaf
Insertion	Leaf split if full
Deletion	Borrow or merge
Balance	Equal leaf depth
Height	Logarithmic

Lecture Summary

- B+ trees are balanced multiway search trees.
- Internal nodes store separator keys.
- Leaf nodes contain indexed entries.
- Leaves are linked in sorted order.
- Searching reaches the leaf level.
- Leaf splits copy separators upward.
- Internal splits promote separators.
- Deletion may require borrowing or merging.
- B+ trees efficiently support range queries.

Questions for Revision

1. Define a B+ tree.
2. Explain internal and leaf node organization.
3. Differentiate B-trees and B+ trees.
4. Explain B+ tree searching.
5. Demonstrate insertion with a leaf split.
6. Explain internal node splitting.
7. Describe borrowing during deletion.
8. Explain merging during deletion.
9. Discuss range searching.
10. Analyze B+ tree operation complexity.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 11: B+ Trees.
2. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*.
3. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*.

Instructional Note

The diagrams and C examples use a consistent maximum-three-keys convention for illustration.

Lecture : Graph Representation

- Introduction to graphs.
- Graph terminology.
- Directed and undirected graphs.
- Weighted and unweighted graphs.
- Adjacency matrices.
- Adjacency lists.
- Graph representation in C.
- Complexity comparison.

Student Feedback

Share Your Feedback



Thank you!